

Lab 3: Supplementary Information

CS 6264-OCY

Overview

- Introduction
- Executing code in Android as root
 - Creating OTA package
 - Packing OTA package
 - Code execution on Android
- Gaining a root shell
 - Reverse shells
 - Launching the attack
- The dangers of being root in Android
 - Check if device is rooted
 - Gaining access to device location
- Further Reading

Introduction

- There are generally three ways to gain root execution in Android
 - Privilege escalation
 - Supply-chain-like attack
 - Replacing the OS
- In this lab, we will focus on the supply-chain-like attack
 - Sort of like a supply-chain attack (think SolarWinds)
 - We rely the code that updates the OS to inject our own code to Android, and gain root using code injection

Over-The-Air (OTA) Attack

- Most Android devices have a secondary OS called the **recovery OS**
- Usually used to recover or update the **Android OS**
- If you were acting as root for this **OS**, you could be able to mount the partition with the **Android OS** installed on it and make changes to files in the **Android OS** (such as initialization files)
- Updates for Android are usually transferred as a package OTA (i.e. downloaded from the manufacturer) and applied to the **Android OS** by the **recovery OS**, extracting the package and running the updater script as root
- If we can inject our own code to this updater script and have the **recovery OS** run it, we would be able to modify the **Android OS**

Executing Code in Android as root

Creating OTA package

- For this part of the lab, we will not require you to gain root privilege on an Android OS when it is initialized
- All we want is for you to prove that you can run code in Android as root when it is initialized
- A simple way to prove this is to run a simple shell script (let's call it `dummy.sh`) that will write data to a directory that is only writable by root, like so:

```
echo hello > /system/dummy
```

Creating OTA package (cont.)

- The lab instructions have more details about OTA package, but most importantly there are two files to keep track of:
 - `META-INF/com/google/android/update-binary` which will execute `updater-script`
 - `META-INF/com/google/android/updater-script` which describes the actions to be executed in order to update Android
- We will need to write our own `updater-script`

Creating OTA package (cont.)

- When Android boots, it runs a script called `/system/etc/init.sh` as root
- Thus, we will want our `updater-script` to do the following:
 1. [Copy](#) our `dummy.sh` script **included in the package** to Android
 2. [Make sure dummy.sh is executable](#)
 3. Inject the contents of `dummy.sh` into `/system/etc/init.sh` (see page 8 of the lab manual for how to do this)
- Keep in mind that when you are in the recovery OS, the contents of the Android OS are found in `/android/` (this may be useful for when you copy your script to the Android OS or when you are injecting your dummy script code into `init.sh`)

Creating OTA package (alt.)

- Alternatively, we can run arbitrary code without injecting code into `init.sh`
- This is achieved by injecting our own code into the bootstrapping process, which is found in the Guidelines section of the lab write-up)
- Essentially, somewhere along the bootloading process, a binary called `app_process32` or `app_process64` will be executed as root depending on the architecture
- If our updater-script can overwrite `app_process` and execute our own code first, and then execute the original `app_process` after, we will be executing arbitrary code as root

Creating OTA package (alt.)

- The code for the new `app_process` can be found on page 9 of the lab manual
- Note that all we are doing is creating a new file called `dummy2`
- Next, we follow instructions on page 10 to compile the new `app_process`
- Finally, we want to write an updater-script that will do the following:
 1. Rename the original `app_process` to `app_process_original`
 2. Copy the new `app_process` binary **from the OTA package folder** to the location of the original and **renaming it to the original name**
 3. Make sure the new binary is executable

Packing the OTA package

- OTA packages are accepted as zip files
- Thus, for our OTA package to be accepted by the recovery OS, we will need to zip it using the [zip command](#)
- Page 8 in the lab manual has more information

Gaining code execution

- Since we don't have a “real” recovery OS, we will be simulating the actions performed by the recovery OS:
 - [Unzip](#) the OTA package
 - Run `META-INF/com/google/android/update-binary`
- After running the binary, we should be able to see the results of whether or not the dummy script ran by checking the `/system` directory after we boot the Android OS

Getting a root shell

Introduction

- Now that we can execute arbitrary code as root, let's try and get an interactive shell so we can run arbitrary commands as root anytime we want, not just when the phone is booted
- Essentially, all we want to do is to redirect our commands as a regular user to the root user's standard I/O file descriptors
- We can do this by using a reverse shell

Reverse shells

- Requires a listener and a client
- Listener will be run as root and forward access to the standard I/O file descriptors to the client when the client connects
- Thus, the client will receive the reverse shell and can interact with the root shell as root
- More about file descriptors is covered by pages 11-14 in the lab manual

Launching the attack

- We have provided code for you to implement a reverse shell listener and client
- How can the listener run as root? Maybe the slides from the previous section might help you set up the listener (or comments found in mydaemonsu.c)
- The client is to be run by the user, so how can an Android user run a normal program? Simply running the compiled program from the terminal of course!

The dangers of being root in Android

Introduction

- There are many nice things that you can do as a root user on an Android device, but there can be drawbacks as well
- One of which is a malicious application gaining access to your device's location without asking for permission
- Your task is to make one of these applications

Checking if device is rooted

- We have provided a utility `.java` file called `RootUtil` to first check if the device is rooted
- There are a few ways you can heuristically check if an Android device is rooted:
 - checking the BUILD tag for Android Test-Keys
 - checking if superuser binary exist in some possible path in the system
 - checked if the user has a UID of 0 by executing `su` and `id` commands
- Methods of implementation can be [found online](#) (try searching something like “check if android device is rooted”)
- Others can include whether or not `app_process` has been hijacked

Getting access to location information

- The main application code is found in `MapsActivity.java`
- Here, you will first check if the device is not rooted, and if it isn't, request the permission called `ACCESS_FINE_LOCATION`
 - You can find out how to request permissions using the [Android documentation](#) or [Android code examples](#)
- If it is rooted, you can directly grant permissions as root in the root shell
 - Permissions in Android Debugger are granted using [package manager](#), but since you are already in the Android shell, you won't need to use `adb shell` in your command
 - You can become root by [calling su](#) ([manpage here](#))
 - Knowing these two things, how can you grant yourself permissions as root? Hint: you can [chain *nix commands together with a semicolon](#)

Further Reading

Useful Manpages

cp

<https://www.man7.org/linux/man-pages/man1/cp.1.html>

chmod

<https://linux.die.net/man/1/chmod>

mv

<https://linux.die.net/man/1/mv>

su

<https://man7.org/linux/man-pages/man1/su.1.html>

zip

<https://linux.die.net/man/1/zip>

unzip

<https://linux.die.net/man/1/unzip>

Root Shells

Understanding Root Shells

<https://www.netsparker.com/blog/web-security/understanding-reverse-shells/>

The Lab Manual

<https://gatech.instructure.com/courses/163506/files/folder/Lab%20Files/Lab%2006?preview=21392915>

Android Documentation

Request App Permissions

<https://developer.android.com/training/permissions/requesting#java>

Permissions App Example

<https://github.com/android/permissions-samples/blob/main/RuntimePermissionsBasic/>

[Application/src/main/java/com/example/android/basicpermissions/MainActivity.java](#)

ADB Package Manager

<http://adbcommand.com/adbshell/pm>

Allow Super User permission programmatically

<https://stackoverflow.com/questions/36935771/allow-super-user-permission-programmatically>

Source code of an Android Application that checks if you are rooted

<https://github.com/scottyab/rootbeer/blob/master/rootbeerlib/src/main/java/com/scottyab/>

[rootbeer/RootBeer.java](#)